Worksheet No. - 3

Student Name: Ankush kumar **UID: 25MCA20305**
Branch: MCA **Section/Group: 1A**
Semester: 2nd **Date of Performance: 27th Jan, 2026**
Subject Name: TECHNICAL TRAINING **Subject Code:**25CAP-652

Aim/Overview of the practical:

To implement conditional decision-making logic in PostgreSQL using **IF–ELSE constructs** and **CASE expressions** for classification, validation, and rule-based data processing.

Objective:

- To understand conditional execution in SQL
- To implement decision-making logic using CASE expressions
- To simulate real-world rule validation scenarios
- To classify data based on multiple conditions
- To strengthen SQL logic skills required in interviews and backend systems

Input/Apparatus Used:

- **PostgresSQL**
- **pgAdmin**

Procedure/Algorithm/Code :

```
/* ========================================================
Step 1: Classifying Data Using CASE Expression
======================================================== */


CREATE TABLE schema_audit (
    schema_name VARCHAR(50),
    violation_count INT
);
```

```
INSERT INTO schema_audit VALUES
('auth_schema', 0),
('user_schema', 2),
('payment_schema', 5),
('admin_schema', 9),
('log_schema', 14);

SELECT
    schema_name,
    violation_count,
    CASE
        WHEN violation_count = 0 THEN 'No Violation'
        WHEN violation_count BETWEEN 1 AND 3 THEN 'Minor Violation'
        WHEN violation_count BETWEEN 4 AND 7 THEN 'Moderate Violation'
        ELSE 'Critical Violation'
    END AS violation_level
FROM schema_audit;

/* ========================================================
Step 2: Applying CASE Logic in Data Updates
======================================================== */

ALTER TABLE schema_audit
ADD COLUMN approval_status VARCHAR(30);

UPDATE schema_audit
```

```
SET approval_status =
    CASE
        WHEN violation_count = 0 THEN 'Approved'
        WHEN violation_count BETWEEN 1 AND 5 THEN 'Needs Review'
        ELSE 'Rejected'
    END;

SELECT * FROM schema_audit;

/* ============================================================
Step 3: Implementing IF-ELSE Logic Using PL/pgSQL
============================================================ */

DO $$
DECLARE
    v_count INT := 6;
BEGIN
    IF v_count = 0 THEN
        RAISE NOTICE 'Schema Status: Approved';
    ELSIF v_count <= 5 THEN
        RAISE NOTICE 'Schema Status: Needs Review';
    ELSE
        RAISE NOTICE 'Schema Status: Rejected';
    END IF;
END $$;

/* ============================================================
```

```
Step 4: Real-World Classification Scenario (Grading System)
============================================================ */

CREATE TABLE student_results (
    student_id INT,
    student_name VARCHAR(50),
    marks INT
);

INSERT INTO student_results VALUES
(1, 'Amit', 92),
(2, 'Riya', 78),
(3, 'Rahul', 64),
(4, 'Sneha', 48),
(5, 'Karan', 85);

SELECT
    student_name,
    marks,
    CASE
        WHEN marks >= 90 THEN 'A'
        WHEN marks >= 75 THEN 'B'
        WHEN marks >= 50 THEN 'C'
        ELSE 'D'
    END AS grade
FROM student_results;
```

```
/* ============================================================
Step 5: Using CASE for Custom Sorting
============================================================ */


SELECT
    schema_name,
    violation_count,
    approval_status
FROM schema_audit
ORDER BY
    CASE
        WHEN violation_count = 0 THEN 1
        WHEN violation_count BETWEEN 1 AND 3 THEN 2
        WHEN violation_count BETWEEN 4 AND 7 THEN 3
        ELSE 4
    END,
    violation_count DESC;
```

Output:

	schema_name character varying (50)	violation_count integer	violation_level text
1	auth_schema	0	No Violation
2	user_schema	2	Minor Violation
3	payment_schema	5	Moderate Viol...
4	admin_schema	9	Critical Violati...
5	log_schema	14	Critical Violati...

Step 1

	schema_name character varying (50)	violation_count integer	approval_status character varying (30)
1	auth_schema	0	Approved
2	user_schema	2	Needs Review
3	payment_schema	5	Needs Review
4	admin_schema	9	Rejected
5	log_schema	14	Rejected

Step 2

```
NOTICE:  Schema Status: Rejected
DO

Query returned successfully in 72 msec.
```

Step 3

Step 4

	student_name character varying (50)	marks integer	grade text
1	Amit	92	A
2	Riya	78	B
3	Rahul	64	C
4	Sneha	48	D
5	Karan	85	B

	schema_name character varying (50)	violation_count integer	approval_status character varying (30)
1	auth_schema	0	Approved
2	user_schema	2	Needs Review
3	payment_schema	5	Needs Review
4	log_schema	14	Rejected
5	admin_schema	9	Rejected

Step 5

Learning outcomes (What I have learnt):

1. I learned how to use **searched CASE expressions** to classify real-world data (such as schema violations) into meaningful categories for compliance and reporting purposes.
2. I understood how **CASE logic can be used inside UPDATE statements** to automate approval decisions directly at the database level, reducing the need for application-side logic.
3. I learned to implement **procedural control flow using PL/pgSQL (IF–ELSIF–ELSE)**, which helped me understand how backend validation and decision-making can be handled within the database.

4. I practiced applying **conditional logic for data categorization**, such as grading students based on marks, which is a common real-world and interview-oriented SQL use case.
5. I learned how to use **CASE in ORDER BY clauses** to apply custom priority-based sorting, which is useful for dashboards, reports, and severity-based listings.
6. I gained confidence in combining **DDL, DML, and procedural SQL** to solve structured, real-world problems using PostgreSQL.